Week9_group&indivisual_assignment_Sulaiman (Stan) Abu Romman

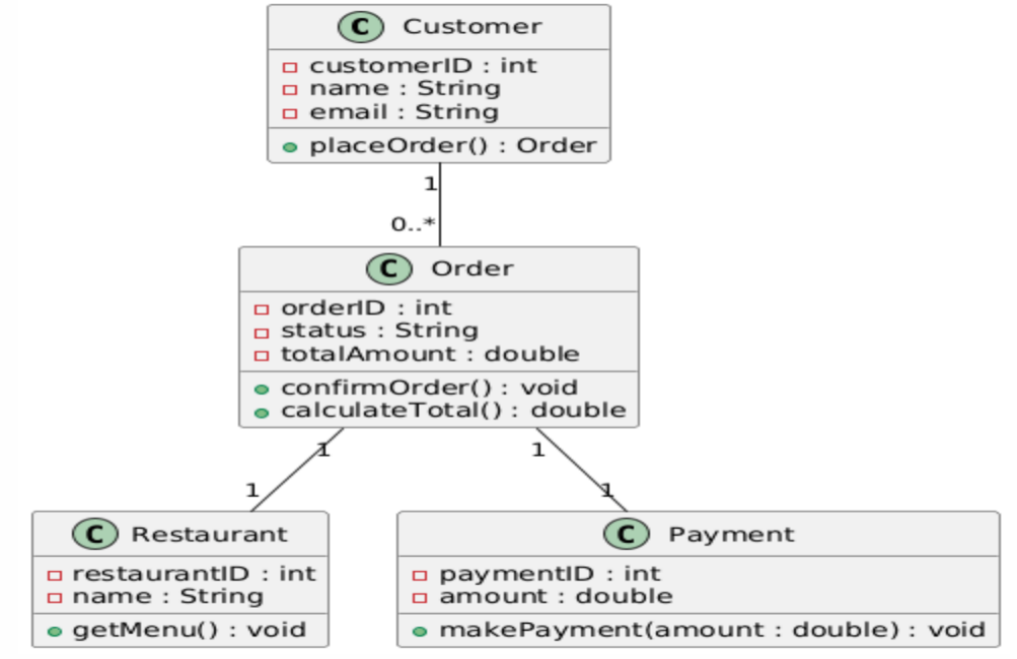
Part 1 – Design Class Diagram (DCD)

A Design Class Diagram (DCD) is a structural UML diagram used in the design phase of software development. It represents software classes, their attributes, operations, and relationships. Unlike a conceptual class diagram, a DCD includes technical details such as data types, method signatures, visibility, and navigability.

A DCD is derived from the domain model and refined using sequence diagrams. The messages exchanged in sequence diagrams become operations in the classes of the DCD. Associations in the DCD represent object references needed to support system behavior.

Design Class Diagram – Stomach Saver

The following Design Class Diagram represents the main software classes of the Stomach Saver online food ordering system. The diagram is based on the existing domain model and the sequence diagrams created in previous weeks. It includes customers placing orders, restaurants providing menus, and payments being processed.



Week9_individual_assignment_Sulaiman (Stan) Abu Romman

Part 2 – Individual Research Task: GRASP

GRASP stands for General Responsibility Assignment Software Patterns. It is a set of guidelines used in object-oriented design to assign responsibilities to classes in a clear and maintainable way. GRASP helps developers decide which class should handle which responsibility based on principles rather than intuition.

The main GRASP techniques include Information Expert, Creator, Controller, Low Coupling, High Cohesion, Polymorphism, Indirection, and Pure Fabrication. For example, the Information Expert principle states that responsibility should be assigned to the class that has the necessary information to fulfill it. The Controller pattern suggests assigning system event handling to a dedicated controller class.

The purpose of GRASP in software development is to improve software quality by promoting well-structured, understandable, and maintainable designs. By applying GRASP principles, developers reduce dependencies between classes, improve flexibility, and make systems easier to extend and maintain. GRASP is especially useful when transforming analysis models, such as use cases and sequence diagrams, into concrete design models like Design Class Diagrams.